第7章　コード最適化

本章では，中間コードのレベルで意味を保ちながらより高速なコードに変換する手法について述べる．ソースコードから構文的に生成された中間コードは不要な繰り返しや変数間のコピーが多く生成されるため，制御フローとデータフローを中間コードを解析して同じ効果をもつより効率的なコードに変換する．この過程を**コード最適化 (code optimization)** と呼ぶ．さらに，最近のコンパイラで用いられている　**静的単一代入形式 (Static single assignment, SSA)** を用いた最適化手法を紹介する．SSA の特徴は，その名前の通り，変数に対する代入を一度に制限して上書きしない．このことによって，変数がより多く必要になる一方，変数の名前によってデータフローを追跡することができ，基本ブロックを越えた最適化がより明確に可能になる．

7.1　基本ブロックとフローグラフ

3 番地コードレベルの抽象度を持つ言語の命令の系列に対して，以下の条件を満たす系列を**基本ブロック**という．

実行制御がその系列の先頭から最後まで，条件付きジャンプおよび無条件ジャンプは最後の場所でのみ出現し，他の命令からのジャンプ先は必ず系列の先頭しかない．

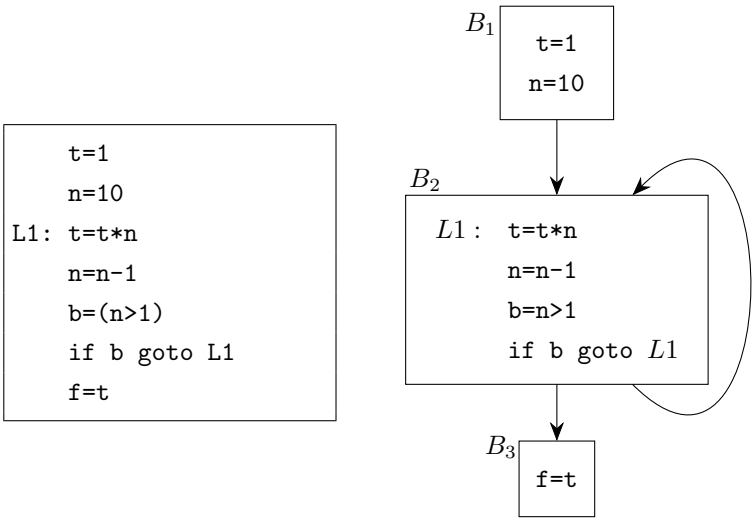


図 7.1: 3 番地コードとその CFG

基本ブロックでは，すべての命令の前提としてそれより前にある命令が必ず実行されている．3

番地コードから構成される基本ブロック内では，参照される変数について，それより前のブロック内で定義されていれば，最後に定義されたところの値を持つことがわかる．そうでなければ，先頭にジャンプしているブロック内の値が引き継がれる．

基本ブロックを頂点として，制御のフローを有向グラフで表した構造を**フローグラフ（あるいは，コントロールフローグラフ，CFG）**と呼ぶ．基本ブロック中では，制御の合流や分岐が存在しないので，CFG はプログラムの振舞いを抽象的に表現していることになる．

7.2　局所最適化

局所最適化は，3 番地コードの系列で構成される基本ブロックに対して同じ意味を持つより短くて変数の少ない系列を求める．

7.2.1　局所最適化の例

定数伝播 (constant propagation)

定数畳み込み (constant folding) とも言う．実行によって値が変わらない変数は，プログラムとして値を埋め込んでメモリ上の値の参照を減らす．

```
a=2*3.14   →   a=6.28      定数計算をコンパイル時に行う．
b=a/100    →   b=0.0628    b はコンパイル時に値が決定できる．
```

変数の値を参照する場合には，メモリから値を読み出す必要があるが，定数にすると値自体をコードとして生成できるので，高速になる．

共通部分式削除

同じ部分式で同じ計算の繰り返しをまとめて，計算の繰り返しを防ぐ．

```
x=a*b        x=a*b
y=x+c        y=x+c
d=a*b   ⇒    d=x
e=d-c        e=d-c
```

以下の系列では，同じ形の部分式 b+c があるが，a と e に対する代入で値が更新されているので，e=a のような消去はできない．

```
a=b+c        a=b+c
b=b-d        b=b-c
c=c+d        c=c+d
e=b+c   ⇏    e=a
```

コピー伝播

`a=b` のような変数のコピーがある場合，a と b は同じ値を保持しているので，a を代入する必要はない．

```
a=b          a=b
c=a+d   ⇒    c=b+d
e=e-a        e=e-b
```

このブロックの後で，`a` が使われることがなければ，`a=b` は消去しても構わない．基本ブロックで用いられる変数については，データフロー解析などの手法を用いて求めることができる場合がある．不要コードの消去 (Dead-code elimination) によってより高速にすることができる．

代数的性質の利用

```
a+0          ⇒   0
b*1          ⇒   b
1+(c+2)      ⇒   3+c
d*(1+e/d)    ⇒   d+e
x*2          ⇒   x+x
x/5          ⇒   x*0.2
```

演算の単位元などによる簡単化，分配則による簡単化などによって式を短縮化する．さらに命令を低レベルのマシン語に変換するときに，除算よりも積算，積算よりも和算の方がサイクルが少ないので，より軽い命令に変更することで高速化が期待できる．

局所最適化については，個々の最適化について大きな効果は期待できない．しかし，基本ブロックがループによって繰り返し実行されるとプログラム全体では大きな効果が期待できる．

7.2.2 基本ブロックの DAG 表現

基本ブロックを有向非巡回グラフ (Directed Acyclic Graph, DAG) を用いて表すことによって，各 3 番地コードにおける参照関係を表現できる．参照関係を壊さない等価なグラフの変形によって最適化を行う．基本ブロックに対して，以下のアルゴリズムによって DAG を生成する．

アルゴリズム 7.1. *DAG* の頂点が始点となる辺を持たない頂点を葉頂点と呼ぶ．葉頂点には，変数または定数のラベルをつける．葉以外の頂点には，演算 θ と，演算の結果が収納される変数のラベルのリストをつける．

1. 基本ブロックに入る前に定義された値を参照する変数名と定数値に対して，葉頂点を生成し，変数 x に対して，x_0 を頂点にラベルづけし，$[x]$ を変数リストとする．定数 c に対しては，c を頂点にラベル付けする．

2. 基本ブロック内のすべての代入文について上から順に以下の操作を適用する．

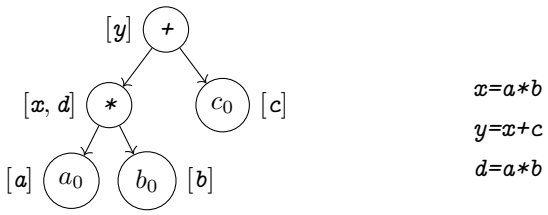
(a) 代入文 $X = A\theta B$ に対して，A,B が変数であるときは変数ラベルに持つ頂点を v_A, v_B とする．定数の場合は定数をラベルにもつ v_A, v_B を選ぶ．

(b) X を変数リストに含む頂点があれば，その変数リストから X を削除する．

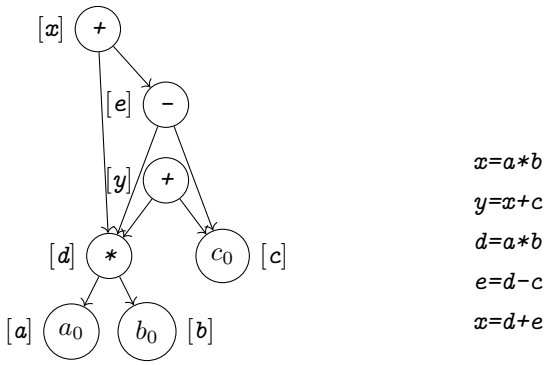
(c) v_A, v_B を子に持ち，θ を演算ラベルとするノード v_θ がすでに存在するならば，v_θ の変数リストに X を追加する．v_θ が存在しないならば，v_θ を新たに生成して，v_A, v_B を子ノードとする辺を追加し，θ を v_θ の演算ラベルとし，$[X]$ を変数リストとする．

(d) $X = A$ の場合は，すでに存在する頂点の変数リストに X が含まれれば，消去した上で，v_A の変数リストに X を追加する．

例 7.1. 以下にアルゴリズムで生成される *DAG* を示す．５行の基本ブロックにおいて，最初の３行までで，まず *(a)* に示す *DAG* が生成される．



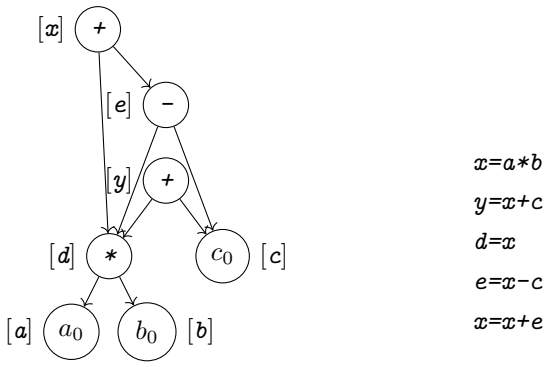
(a) ３行目までで生成される *DAG*



(b) 5 行目までで生成される *DAG*

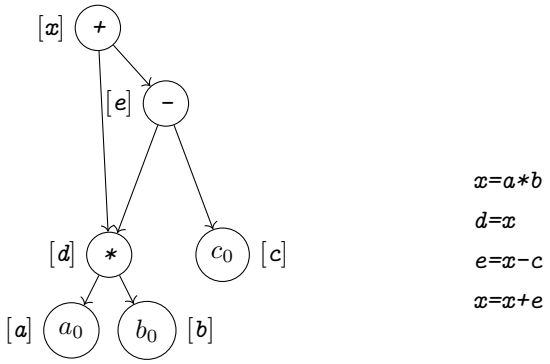
４行目の命令でさらに e に対するノードが新たに生成され，５行目では x が更新されるため，既存の変数リストから x を消去して新しく一番上のノードに x を変数リストに追加している．

例 *7.1* の *(a)* において，$[x, d]$ のように複数の変数が一つのノードにつけられる場合は，共通部分式が存在するということであるので共有部分式の消去が適用できる．さらに，コピー伝播を適用すると *(c)* のようになる．



(c) 共有部分式とコピー伝播の適用

この基本ブロックの後で y が使われない場合は，$[y]$ を変数リストにもつ頂点とその辺を消去しても意味は変わらない．これに対して，d が使われない場合は，$[d]$ の頂点を消すことはできない．これは，x が d の頂点に依存しているためである．



(d) 不要コード消去の適用

7.3 大域最適化

複数の基本ブロックにわたる最適化を大域的最適化とよぶ．この最適化を行うために，フローグラフの構造にマッチする構造を見つける方法について示す．

7.3.1 意味等価な変換

プログラムにおいて，全体の１，２割の部分が，実際の実行の８，９割の時間を占める．これは，プログラムの一部がループ構造になって繰り返し実行されることによる．繰り返しに含まれる基本ブロックを局所最適化によって最適化することによって大きな効果が期待できることは述べた．これに加えて，ループ構造として最適化する手法について示す．

ループ構造の最適化は，制御フローグラフに依存するため，ソースコードでも３番地コードでもほぼ変わらない．以下では，C 言語プログラムの一部で最適化の手法について示す．

ループ最適化

ループ不変コードの移動　ループ内で値が変化しない変数をループの外に移動し，計算に影響しない命令が何度も実行されずに同じ意味を表現するようにする．

```
i=1;                              i=1;t=a/b;
do {                              do {
    c=a/b;x=i*c; .... i++;   ⇒        c=t;x=i*c;...i++;
} while(i<n);                     } while(i<n);
```

do ... while で，a, b のいずれも更新されないとき，c はループ実行で変化しないため，ループ外で値を一度代入する同じ計算となる．

```
                                  if(n>1) {
                                      t=a/b;
for(i=1;i<n;i++) {                    for(i=1;i<n;i++) {
    c=a/b;x=i*c;...           ⇒           c=t;x=i*c;...
}                                     }
                                  }
```

一般的な for ループでは，条件によってループの内容が実行されないこともあるため，不変コードはループが実行された場合だけ，実行されるように変換する必要がある．

ループ内演算の強さ軽減

```
                                  t=0;
for(i=1;i<n;i++)                  for(i=1;i<n;i++) {
    a=a+b*i;                  ⇒       t=t+b;a=a+t;
                                  }
```

帰納変数の消去

```
t=b;
for(i=1;i<n;i++) {                m=n*b;
    a=a+t;t=t+b;              ⇒   for(t=b;t<m;t=t+b)
}                                     a=a+t;
```

ループ融合

```
for(i=1;i<n;i++)                  for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;                    a[i+1]=a[i]+i;
                              ⇒
for(i=1;i<n;i++)                      b[i]=a[i]+i;
    b[i]=a[i]+i;                  }
```

ループ分解

```
for(i=1;i<n;i++) {                for(i=1;i<n;i++)
    a[i+1]=a[i]+i;                    a[i+1]=a[i]+i;
                              ⇒
    b[i]=a[i]+i;                  for(i=1;i<n;i++)
}                                     b[i]=a[i]+i;
```

ループ展開

```
for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+b[i-1];
}
```

⇒

```
for(i=1;i<n;i++) {
    a[i]=a[i-1]+i;
    b[i]=a[i]+b[i-1];a[i+1]=a[i]+1;
    b[i+1]=a[i+1]+b[i];
}
```

ソフトウェアパイプライン

```
for(i=1;i<1000;i++) {
    a[i]=a[i-1]+1; // Ai
    b[i]=a[i]+b[i-1]; // Bi
    c[i]=b[i]+2; // Ci
}
```

⇒

```
A1;
B1; A2;
for(i=3;i<1000;i++) {
    Ci-2;Bi-1;Ai;
}
C998;B999;
C999;
```

手続き呼び出し最適化

インライン展開 手続き呼び出しは，スタック上に局所環境を作成し，リターンの際にもスタック上の局所環境をクリアする必要がある．局所環境を作成するほどでもない単純な手続き呼び出しでは，スタック操作を行わなくてもよい場合もあり，ループ構造内では大きな高速化の効果が期待できる．

```
int square(int x) {
    return x*x;
}
y = square(a);
```

⇒

```
y = a * a;
```

c 言語では，関数の前に `inline` を付加することでスタックを用いずに呼び出しを行う．ただし，`inline` をつけた場合は，実際にインライン展開するかどうかはコンパイラで判断する．必ず展開する場合は，`#define` でマクロ定義すれば呼び出しと同様の形式でプログラムを記述できる．

末尾再帰 再帰呼び出しは，呼び出し定義の中で自分を呼び出すことである．定義の中の呼び出しが定義の最後の命令であるとき，関数からリターンしたあともとの環境の変数は参照されないので，環境を復元する必要がない．従って，インラインでない関数呼び出しを行う場合であっても，毎回，局所環境を作成する必要がなく，最初の１回目の呼び出しで作成した領域を上書きすればよい．変換としては，再帰呼び出しを goto に置き換えることで高速化が可能である．

```
int sum_tail(int n, int acc) {
    if (n == 0) {
        return acc;
    }
    return sum_tail(n-1,acc+n);
}
```

⇒

```
int sum_tail(int n, int acc) {
L:  if (n == 0) {
        return acc;
    }
    acc+=n; n--;
    goto L;
}
```

goto によるループ構造を goto なしで記述すると以下のプログラムと等価である．

```
int sum(int n,int acc) {
    while(n!=0) {
        acc+=n; n--;
    }
    return acc;
}
```

7.3.2　データフロー解析

プログラムの実行意味が状態遷移に基づいて決まるとすると，プログラムの状態を決定する変数の実行に従った更新（**データフロー**）を解析することでプログラムの実行意味を捉えることが可能になる．基本ブロックを越えて最適化を行うためには，フローグラフ上で各基本ブロックにどの変数が入力されてどの変数を出力するか，ということを解析して，代入された値がどこまで参照として有効か（定義到達解析），以降の実行でどこまでその変数が用いられるか（生存解析），同じ値を計算する式の検出（利用可能式解析）などの解析手法が用いられる．このような解析はプログラムを実行することなく行われるため，**静的解析**という．この解析において，一般にプログラムの成魚がループを持つことから変数の更新は自身の更新に依存するため，この解析においてはデータフロー方程式を解く必要が生じる．

定義到達解析

定義到達解析は，v=E という形の 3 番地コードは v を定義しているとし，ここで定義された v の値がどこまで変更されずに到達するか，ということを解析する．v が別の命令で上書きされるとき，先の定義は kill される．各定義の 3 番地命令にラベル d_i をつけ，基本ブロック B ごとに以下の定義ラベルの集合を定める．

- $Gen[B]$: B のなかで定義され，上書きされない定義の集合
- $Kill[B]$ B のなかで上書きされる定義の集合
- $In[B]$ B が実行される際に参照する変数に対する定義の集合
- $Out[B]$ B が実行された後に有効な定義の集合

ここで Gen, $Kill$ は定義の実行順は関係なく，定義集合全体に対して定める．

基本ブロックから構成されるフローグラフが与えられたとき，以下の関係がある．

$$In[B] = \cup_{P \in \mathsf{Pred}(B)} Out[P] \tag{7.1}$$

$$Out[B] = Gen[B] \cup (In[B] \backslash Kill[B]) \tag{7.2}$$

ここで，$\mathsf{Pred}(B)$ は，フローグラフにおいて，B の先行頂点 (predecessor) の集合を表す．基本ブロック内では，制御の分岐はない．基本ブロックには，その直是のブロックから出力されるフロー

が入力され，ブロックからの出力フローは，ブロック内で新たに発生した定義と，入力定義のうち kill されなかった定義となる．

図**??**のコードにおいて以下のように定義のラベル付けをする．

```
d1 :        t=1
d2 :        n=10
d3 :  L1 :  t=t*n
d4 :        n=n-1
d5 :        b=(n>1)
            if b goto L1
d6 :        f=t
```

B_i に対する Gen, $Kill$ は以下のようになる．

$Gen[B_1] = \{d_1, d_2\}$, $Kill[B_1] = \{d_3, d_4\}$

$Gen[B_2] = \{d_3, d_4, d_5\}$, $Kill[B_2] = \{d_1, d_2\}$

$Gen[B_3] = \{d_6\}$, $Kill[B_3] = \varnothing$

In, Out については以下の関係がなりたつ．

$In[B_1] = \varnothing$, $Out[B_1] = Gen[B_1] = \{d_1, d_2\}$

$In[B_2] = Out[B_1] \cup Out[B_2]$, $Out[B_2] = \{d_3, d_4, d_5\} \cup (In[B_2] \backslash \{d_1, d_2\})$

$In[B_3] = Out[B_2]$, $Out[B_3] = \{d_6\} \cup (In[B_2] \backslash \{d_6\})$

$In[B_2]$ についての式を代入すると，

$$In[B_2] = \{d_1, d_2\} \cup (\{d_3, d_4, d_5\} \cup (In[B_2] \backslash \{d_1, d_2\}))$$

となる．$In[B_2] = X$ とおくと，$X = \{d_1, d_2, d_3, d_4, d_5\} \cup (X \backslash \{d_1, d_2\})$ を満たす X を求める必要がある．これを解くには，$X_0 = \varnothing$ として，$X_{i+1} = \{d_1, d_2, d_3, d_4, d_5\} \cup (X_i \backslash \{d_1, d_2\})$ で，$X_{i+1} = x_i$ となるまで繰り返す．

$X_1 = \{d_1, d_2, d_3, d_4, d_5\}$, $X_2 = \{d_1, d_2, d_3, d_4, d_5\}$ となるので $X = \{d_1, d_2, d_3, d_4, d_5\}$ が条件を満たす．したがって，$Out[B_2] = \{d_3, d_4, d_5\}$，$out[B_3] = \{d_3, d_4, d_5, d_6\}$ である．

この解析によってこのプログラムが終了した時点での，t,n の有効な定義は，d_3 および d_4 であり，d_1, d_2 の定義は伝播しないということがわかる．また，$In[B_2]$ において，t, n に対する定義は一意的に定まらないので，B_2 中でそれらの変数を定数に置き換えることはできないこともわかる．

生存変数解析

基本ブロック B において，$In[B]$ に含まれる定義の変数が Kill される前に参照されるとき，その定義は B で実際に使用される．この変数の集合を $Use[B]$ とする．B の中で定義される変数の集合を $Def[B]$ とすると，以下の関係が成り立つ．

$$In[B] = Use[B] \cup (Out[B] \backslash Def[B]) \tag{7.3}$$

$$Out[B] = \cup_{S \in \mathsf{Succ}(B)} In[S] \tag{7.4}$$

ここで，Succ(B) はフローグラフでの B の後続頂点集合である．

図 7.1 について，生存変数解析は以下のようになる．

$Use[B_1] = \varnothing$, $Def[B_1] = \{t, n\}$

$Use[B_2] = \{t, n\}$, $Def = \{t, n, b\}$

$Use[B_3] = \{t\}$, $Def[B_3] = \{f\}$

(7.3)(7.4) を適用する．

$In[B_1] = Out[B_1] \backslash \{t, n\}$, $Out[B_1] = In[B_2]$

$In[B_2] = \{t, n\} \cup (Out[B_2] \backslash \{t, n, b\})$, $Out[B_2] = In[B_2] \cup In[B_3]$

$In[B_3] = \{t\} \cup (Out[B_3] \backslash \{f\})$,$Out[B_3] = \varnothing$

In, Out は出口から計算できる．$In[B_3] = \{t\}$ $Out[B_2] = In[B_2] \cup \{t\}$, $In[B_2] = \{t, n\} \cup ((In[B_2] \cup \{t\}) \backslash \{t, n, b\})$ $In[B_2] = X$ とすると，$X = \{t, n\} \cup (X \backslash \{n, b\})$ となる．$X_0 = \varnothing$ として $X_{i+1} = \{t, n\} \cup (X_i \backslash \{n, b\})$ に代入すると $X_1 = \{t, n\}$, $X_2 = \{t, n\}$ となる．よって，$Out[B_2] = \{t, n\}$, $Out[B_1] = \{t, n\}$, $In[B_1] = \varnothing$ と決定できる．

生存変数解析によって，基本ブロックの実行の後で使用されない変数を特定できる．この解析に基づいて使われない変数の計算を基本ブロックから消去することができる．

図 7.1 に対する利用可能式解析は以下のようになる．

$Gen[B_1] = \varnothing$, $Kill[B_1] = \{$`t*n`, `n-1`, `n>1`$\}$

$Gen[B_2] = \{$`n>1`$\}$, $Kill[B_2] = \{$`t*n`, `n-1`, `n>1`$\}$

$Gen[B_3] = \varnothing$, $Kill[B_3] = \varnothing$

IN,OUT は以下のようになる．

$IN[B_1] = \varnothing$, $OUT[B_1] = \varnothing$

$IN[B_2] = OUT[B_1] \cap OUT[B_2] = \varnothing$ $OUT[B_2] = \{$`n>1`$\}$

$IN[B_3] = OUT[B_2] = \{$`n>1`$\}$, $OUT[B_3] = \{$`n>1`$\}$

実際に利用可能解析は，配列のアドレス計算において `a[i]` のデータにアクセスするのに `a` のアドレス計算に `i*4` を計算する場合などに有効である．

利用可能式解析

v = a + b という代入コードがある場合，この基本ブロックの後で a + b という式が出現しているとき a, b の値が更新されていなければ，計算の重複を避けることが可能になる．変数や定義の代わりに評価式を蓄積する．式はその要素が一つでも更新されると kill される．各基本ブロック B について，B の中において計算された後 B を抜けるまでに kill されることのない式の集合を $Gen[B]$，B の中で kill される式の集合を $Kill[B]$ とする．ここでは，以下の関係が成り立つ．

$$IN[B] = \cap_{\mathsf{Pred}[B]} Out[P] \tag{7.5}$$

$$OUT[B] = Gen[B] \cup (In[B] \backslash Kill[B]) \tag{7.6}$$

7.4 静的単一代入形式と最適化

最近のコンパイラにおける最適化では，静的単一代入形式 (Static single assignment, SSA) を用いる事が多い．SSA 形式では中間コード列の中に各変数の定義（３番地コードの場合は代入の左辺）が一箇所しか現れないという制約を持つ．定義が１つしかないため，どこで定義された変数がどこで参照されるかが，プログラムとして明示される．プログラムを解析すれば，プログラムの構文情報からデータフローを解析することができる．

7.4.1 例

7.4.2 ϕ 関数

7.4.3 SSA 形式への変換

7.4.4 SSA 上での最適化

7.4.5 逆変換